Efficient Multivector Retrieval with Token-Aware Clustering and Hierarchical Indexing*

Silvio Martinico[1,2,*], Franco Maria Nardini[2], Cosimo Rulli[2] and Rossano Venturini[1]

[1]University of Pisa, Largo B. Pontecorvo 3, 56127 Pisa, Italy
[2]ISTI-CNR, Via G. Moruzzi 1, 56124 Pisa, Italy

Abstract

Late-interaction retrieval encoders achieve state-of-the-art effectiveness through fine-grained token-level representations, but their deployment incurs substantial computational and memory costs. Current solutions rely on the well-known κ-means clustering algorithm, which scales poorly with the number of clusters and dataset size, and favours frequent tokens during training while underrepresenting rare, discriminative ones. We introduce Tachiom, a multivector retrieval system that exploits token-level structure to accelerate both clustering and retrieval: by accounting for tokens' distribution during centroid allocation, it easily scales to millions of centroids, enabling highly accurate document scoring using only centroids, and it combines a graph-based index over centroids with an optimized Product Quantization layout for efficient final scoring. On Ms Marco-v1 and LoTTE, Tachiom achieves up to 247× faster clustering than κ-means and up to 9.8× retrieval speedup over state-of-the-art systems, while maintaining comparable or superior effectiveness.

Keywords

Multivector Retrieval, Late Interaction, Clustering, Efficiency

1. Introduction

Late-interaction multivector models, such as ColBERT [2, 3], have emerged as a gold standard for effectiveness in Information Retrieval. By encoding documents as sets of token-level vectors and computing relevance via late interaction, i.e., allowing every query token to interact with every document token, these models capture fine-grained semantic nuances that single-vector dense retrievers often miss. Their deployment, however, remains severely constrained by their computational demands, as storing and searching large sets of token vectors creates massive memory footprints. To mitigate this, state-of-the-art solutions adopt a *gather-and-refine* strategy combined with token quantization [4, 5, 6, 7, 8]: a lightweight search over token vectors first identifies promising candidates, then full MaxSim scores are computed only for the selected ones. Both phases critically depend on a coarse quantizer—a set of centroid vectors that approximate the token embedding space—which provides efficient proxies for locating relevant tokens during gathering and, for compression, decomposes each token vector into a centroid assignment and a residual, compressed with Product Quantization [9] (PQ) or similar techniques.

ColBERTv2 [3] pioneered this centroid-based approach, compressing the residual, and Plaid [5] extended it with centroid interaction and inverted-file filtering. Subsequent work added bitvector prefiltering and optimized PQ variants [6, 10, 11], lightweight architectures [7, 12], and graph indexes over the centroids that avoid exhaustive centroid scoring [8].

Across all these methods, both retrieval quality and compression effectiveness scale directly with the number of centroids. However, standard κ-means—the predominant method for centroid selection—creates a severe bottleneck: it scales poorly with both dataset size and number of clusters and, critically,

IIR 2026: 16th Italian Information Retrieval Workshop, November 12–14, 2026, Rome, Italy
*Extended abstract of [1], SIGIR '26.
*Corresponding author.
silvio.martinico@phd.unipi.it (S. Martinico); francomaria.nardini@isti.cnr.it (F. M. Nardini); cosimo.rulli@isti.cnr.it (C. Rulli); rossano.venturini@unipi.it (R. Venturini)
0009-0005-7280-6147 (S. Martinico); 0000-0003-3183-334X (F. M. Nardini); 0000-0003-0194-361X (C. Rulli); 0000-0002-9830-3936 (R. Venturini)
© 2026 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

it ignores the token structure of multivector embeddings, so that training even moderately large sets of centroids (e.g., 262K) can require tens of hours on multi-core CPUs or expensive GPU resources.

We address this bottleneck by exploiting a key structural property of multivector embeddings: *token identity*. Common tokens (e.g., stopwords) dominate κ-means' loss during training, consuming the majority of centroids, while rare domain-specific tokens—often more discriminative for retrieval—receive minimal representation. We introduce Tachiom (Token-Aware Clustering and Hierarchical Indexing for Optimized Multivector retrieval), which exploits this structure at both the clustering and the retrieval stages. At its core, Tachiom employs Token-Aware Clustering (Tac), which allocates centroids based on token frequency and semantic variance and is provably faster than κ-means, enabling scaling to millions of centroids. Leveraging these high-resolution centroids, Tachiom performs gathering using only centroid-level interactions via an Hnsw graph index [13], bypassing expensive token-level computations entirely, while a cache-optimized PQ layout tailored for late interaction enables efficient refinement. Our Rust implementation is publicly available at `github.com/TusKANNy/tachiom`.

2. Token-Aware Clustering

κ-means minimizes the Within-Cluster Sum of Squares (WCSS), treating all vectors equally, but multivector collections exhibit severe frequency imbalance: analysing two ColBERTv2 collections, we observed that the top 100 most frequent tokens account for 41% of all vectors on Ms Marco-v1 and 45% on LoTTE-pooled. The WCSS loss is thus dominated by high-frequency tokens, which consume the majority of centroids despite contributing little to retrieval quality, while rare tokens—which often carry the most discriminative semantic signal [14]—receive minimal representation, degrading approximation quality precisely where it matters most.

Drawing from classic IR literature, where *Inverse Document Frequency* has long been employed to favor rare, discriminative terms [15, 16], we introduce *Token-Aware Clustering* (Tac), that decouples centroid allocation from uniform WCSS minimization by weighting tokens based on their frequency and semantic variance. It first computes, for each token type, its share of the total centroid budget, then performs an independent κ-means clustering per token type. The budget is distributed through a four-stage pipeline. *Tail Handling* isolates very rare tokens to prevent their complete marginalization, assigning a fixed number of centroids to tokens whose frequency falls below the thresholds μ and τ. For the remaining *active* tokens, *Damped Scoring* allocates κ_j proportionally to the weight $w_j = \sqrt{n_j} \cdot s_j$, where n_j is the frequency of token j and s_j is its component-wise variance: the square-root damping applies diminishing returns to highly frequent tokens, preventing them from monopolizing the centroid budget, while the spread coefficient favors tokens with high semantic variance. *Bounding* then enforces a hard floor ε on κ_j and an upper bound $n_j/\kappa_j \geq \theta$ on the average number of vectors per centroid. Finally, *Budget Reconciliation* redistributes surplus or deficit centroids to exactly match the target budget κ.

Tac thus prioritizes retrieval quality over pure reconstruction error and, instead of optimizing a single global objective over all N vectors with κ centroids, it solves independent subproblems for each token, reducing the cost from $O(I \cdot N \cdot \kappa \cdot d)$ to $O(I \cdot \sum_j n_j \cdot \kappa_j \cdot d)$ for I iterations in d dimensions and enabling effective parallelization, as each token can be processed independently, whereas standard κ-means requires shared access to the full dataset. It can be proved that the resulting speedup is lower-bounded by the ratio between the total weight and the maximum single-token weight, which guarantees a substantial baseline speedup and shows that damped scoring, by reducing the maximum single-token weight, strictly improves the guarantee. Once centroids are computed, we compress the residuals with PQ [9]; because residual magnitudes vary across tokens due to the non-uniform allocation, we normalize them and store their norms separately, obtaining more homogeneous vectors that are easier to compress.

3. Hierarchical Index for Optimized Multivector Retrieval

The speedups achieved by Tac allow us to significantly scale up the number of centroids, which improves the approximation provided by centroids alone and eliminates the need for full token scoring (centroid + residual) during the gather phase, differently from existing approaches [6, 8, 7]. Decoupling gathering from the residual score computation also lets us specialize the refinement stage with a PQ layout optimized for document-level MaxSim evaluation.

Index Structure and Gather Phase. We build an Hnsw [13] graph over the centroid set; each centroid $\mathbf{c}_i$ maintains an inverted list $\mathscr{L}_i$ containing the document IDs of all tokens assigned to it, which enables document-level scoring without accessing individual token vectors. For each query token $\mathbf{q}_i$, we traverse the graph to identify the top-κ_c nearest centroids and iterate over their inverted lists; for documents appearing in multiple lists we retain only the highest centroid similarity, $\tilde{s}_i(d) = \max_{\{j\,:\,d\in\mathscr{L}_j\}}\langle \mathbf{q}_i, \mathbf{c}_j\rangle$, and we accumulate partial scores across all query tokens, $\tilde{S}(q,d) = \sum_i \tilde{s}_i(d)$. This yields an approximate MaxSim score computed from centroids alone. We then truncate the candidate set to the top κ_d documents and apply the *Candidate Pruning* strategy of [17], regulated by a parameter α.

Refine Phase. For the pruned candidate set, we compute full MaxSim scores using both centroids and PQ-compressed residuals. Since gathering is performed at the document level and intermediate token-level scores are never needed, the PQ distance tables need not follow the standard, per-query-token layout, which would force n_q scattered accesses across n_q disjoint tables. We instead reorganize the precomputed distances into a three-level hierarchy—macro-blocks indexed by PQ subspace, centroid blocks indexed by PQ centroid ID, and micro-blocks holding the n_q distances between each query token and that centroid—so that scoring a document token's code retrieves all n_q distances as a single contiguous read. This yields up to a 3.8× speedup in residual distance computation over a standard PQ implementation.

4. Experiments

We experiment on Ms Marco-v1 [18] (8.8M passages, 598M token vectors, 6,980 dev.small queries) in-domain and on LoTTE-pooled [3] (2.4M passages, 266M token vectors, 2,931 search/dev queries) out-of-domain, using ColBERTv2 [3] as the encoder and reporting MRR@10 and Success@5, respectively. Our Rust implementation is built on top of the kANNolo [19] library and runs on an Intel Xeon Silver 4314 CPU @ 2.40GHz; clustering exploits all 64 threads in parallel, while retrieval is executed sequentially on a single core.

Clustering. We compare Tac against two κ-means baselines: Faiss [20], both in its generic AVX2 configuration (comparable to our setup, which does not employ complex and heavily-optimized matrix-multiplication libraries such as Intel MKL [21]) and with MKL enabled, and FastKMeans-rs, an efficient Rust implementation of the FastKMeans library [22]. We set the tail handling thresholds to $\mu = 128$ and $\tau = 256$ and the bounding parameters to $\varepsilon = 4$ and $\theta = 39$; residuals are compressed with $M = 32$ PQ subspaces and 8-bit codes, and we allow a 24-hour budget for the algorithms' running time.

Table 1 reports clustering times, including both centroid computation and assignment of all vectors.

Table 1
Clustering time (minutes) on Ms Marco-v1. "–" indicates that the 24-hour budget is exceeded.

	Number of Centroids					
	131k	262k	524k	1.0M	2.1M	4.2M
FastKMeans-rs	1380	–	–	–	–	–
Faiss	1483	–	–	–	–	–
Faiss w/ MKL	352	617	1265	–	–	–
Tac	**6**	**10**	**17**	**32**	**64**	**102**

Table 2
Average query time (ms) at fixed MRR@10 (Ms Marco-v1) and Success@5 (LoTTE). "–": level not reached.

	Ms Marco-v1			LoTTE		
	39.0	39.3	39.6	67.5	68.0	68.5
Warp	98	98	–	49	–	–
Igp	72	–	–	48	–	–
Emvb	55	56	57	54	55	59
Tachiom	**10**	**14**	**15**	**11**	**14**	**23**

Tac achieves training times orders of magnitude lower than all competing methods, even when they benefit from Intel MKL acceleration, with speedups up to 84× (Faiss with MKL), 247× (Faiss), and 230× (FastKMeans-rs). It clusters nearly 600 million 128-dimensional vectors into 262K clusters—the budget used by Plaid, Emvb, Warp, and Igp—in 10 minutes and scales seamlessly to over 4M centroids in 102 minutes, whereas the 24-hour timeout prevents the competitors from surpassing 131K centroids (Faiss and FastKMeans-rs) and 524K (Faiss with MKL). This efficiency does not come at the price of quality: reranking the top 1,000 candidates retrieved by Splade CoCondenser [23] with the compressed representations (as a proxy for exhaustive search), Tac delivers MRR@10 equal to or higher than κ-means at every centroid budget.

Retrieval. We compare Tachiom against three state-of-the-art multivector retrieval methods, namely Emvb [6], Warp [7] and Igp [8], reporting the best (lowest) average query time at different metric cut-off points. We use $2^{21} \approx 2$M centroids for LoTTE and $2^{22} \approx 4$M for Ms Marco-v1, and build the Hnsw graphs with $M = 32$ neighbors and $ef_c = 1500$. At search time we grid search the number of centroids retrieved per query token $\kappa_c \in \{15, \dots, 120\}$, the maximum number of candidate documents $\kappa_d \in \{250, \dots, 4000\}$ and the pruning threshold $\alpha \in \{0.35, \dots, 0.5\}$, with $ef_s = \frac{3}{2}\kappa_c$; for the competitors we follow the grid search procedures of [17]. All methods use the same residual size, i.e., 32 bytes per token vector.

Table 2 shows that Tachiom is more efficient at all cutoffs, with speedups ranging from 2.5× to 9.8× over the baselines. It is up to 5.5× faster than Emvb, the best available data structure, and matches its peak effectiveness despite employing standard PQ, whereas Emvb relies on computationally expensive PQ variants, namely JMPQ [11]—a supervised method that jointly trains centroids, PQ, and the query encoder—on Ms Marco-v1 and OPQ [10] on LoTTE. Through its careful centroid selection and its ability to scale the number of centroids via Tac, Tachiom rivals both without incurring their additional training overhead.

5. Conclusions and Future Work

We presented Tac, a clustering algorithm that decomposes the global clustering problem into independent per-token subproblems and allocates centroids based on token frequency and semantic variance, and Tachiom, a retrieval architecture that leverages the resulting high-granularity centroids to gather candidates while bypassing expensive token-level interactions. Together, these contributions unlock the practical utility of multivector retrieval for large-scale datasets. Future work will assess the generalizability of Tac beyond ColBERTv2, evaluate Tachiom on datasets substantially larger than Ms Marco-v1, and investigate higher residual compression ratios and further optimizations of the framework.

Acknowledgments

This research was conducted as part of the ENRESFOOD project, funded under the FutureFoodS partnership. The project is co-funded by the Austrian Science Fund (FWF; 10.55776/KIN4661525), The Dutch Ministry of Agriculture, Fisheries, Food Security and Nature (BO-43-219-026), the Italian Ministry of University and Research (MUR), the German Federal Ministry of Research, Technology and Space (031B1717), and the European Union's Horizon Europe research and innovation programme via FutureFoodS, the European Partnership for a Sustainable Future of Food Systems, co-funded under Grant Agreement No. 101136361.

Declaration on Generative AI

During the preparation of this work, the author(s) used Claude in order to: Paraphrase and reword. After using this tool, the author(s) reviewed and edited the content and take(s) full responsibility for the publication's content.

References

[1] S. Martinico, F. M. Nardini, C. Rulli, R. Venturini, Efficient multivector retrieval with token-aware clustering and hierarchical indexing, in: Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR '26, Association for Computing Machinery, New York, NY, USA, 2026, p. 3982–3987. URL: https://doi.org/10.1145/3805712.3809927. doi:10.1145/3805712.3809927.

[2] O. Khattab, M. Zaharia, ColBERT: Efficient and effective passage search via contextualized late interaction over bert, in: Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval, 2020, p. 39–48. doi:10.1145/3397271.3401075.

[3] K. Santhanam, O. Khattab, J. Saad-Falcon, C. Potts, M. Zaharia, ColBERTv2: Effective and efficient retrieval via lightweight late interaction, in: Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, 2022, pp. 3715–3734. doi:10.18653/v1/2022.naacl-main.272.

[4] J. Engels, B. Coleman, V. Lakshman, A. Shrivastava, DESSERT: an efficient algorithm for vector set search with vector set queries, in: Proceedings of the 37th International Conference on Neural Information Processing Systems (NeurIPS), 2023.

[5] K. Santhanam, O. Khattab, C. Potts, M. Zaharia, PLAID: An efficient engine for late interaction retrieval, in: Proceedings of the 31st ACM International Conference on Information & Knowledge Management (CIKM), 2022, p. 1747–1756. doi:10.1145/3511808.3557325.

[6] F. M. Nardini, C. Rulli, R. Venturini, Efficient multi-vector dense retrieval with bit vectors, in: Proceedings of the 46th European Conference on Information Retrieval (ECIR), 2024, p. 3–17. doi:10.1007/978-3-031-56060-6_1.

[7] J. L. Scheerer, M. Zaharia, C. Potts, G. Alonso, O. Khattab, WARP: An efficient engine for multi-vector retrieval, in: Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval, 2025, p. 2504–2512. doi:10.1145/3726302.3729904.

[8] Z. Bian, M. L. Yiu, B. Tang, IGP: Efficient multi-vector retrieval via proximity graph index, in: Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval, 2025, p. 2524–2533. doi:10.1145/3726302.3730004.

[9] H. Jégou, M. Douze, C. Schmid, Product quantization for nearest neighbor search, IEEE transactions on pattern analysis and machine intelligence 33 (2011) 117–28. doi:10.1109/TPAMI.2010.57.

[10] T. Ge, K. He, Q. Ke, J. Sun, Optimized product quantization, IEEE Trans. Pattern Anal. Mach. Intell. 36 (2014) 744–755. doi:10.1109/TPAMI.2013.240.

[11] Y. Fang, J. Zhan, Y. Liu, J. Mao, M. Zhang, S. Ma, Joint optimization of multi-vector representation with product quantization, in: Proceedings of the 11th CCF International Conference Natural Language Processing and Chinese Computing, 2022, p. 631–642. doi:10.1007/978-3-031-17120-8_49.

[12] J. Lee, Z. Dai, S. M. K. Duddu, T. Lei, I. Naim, M.-W. Chang, V. Y. Zhao, Rethinking the role of token retrieval in multi-vector retrieval, in: Proceedings of the 37th International Conference on Neural Information Processing Systems (NeurIPS), 2023.

[13] Y. A. Malkov, D. A. Yashunin, Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs, IEEE Trans. Pattern Anal. Mach. Intell. 42 (2020) 824–836. doi:10.1109/TPAMI.2018.2889473.

[14] K. Sparck Jones, A statistical interpretation of term specificity and its application in retrieval, Taylor Graham Publishing, GBR, 1988, p. 132–142.

[15] S. E. Robertson, Understanding inverse document frequency: on theoretical arguments for idf, J. Documentation 60 (2004) 503–520. URL: https://api.semanticscholar.org/CorpusID:8864928.

[16] G. Salton, A. Wong, C. S. Yang, A vector space model for automatic indexing, Commun. ACM 18 (1975) 613–620. URL: https://doi.org/10.1145/361219.361220. doi:10.1145/361219.361220.

[17] S. Martinico, F. M. Nardini, C. Rulli, R. Venturini, Multivector reranking in the era of strong first-stage retrievers, in: R. Campos, A. Jatowt, Y. Lan, M. Aliannejadi, C. Bauer, S. MacAvaney, A. Anand, Z. Ren, S. Verberne, N. Bai, M. Mansoury (Eds.), Advances in Information Retrieval,

Springer Nature Switzerland, Cham, 2026, pp. 49–65.
[18] T. Nguyen, M. Rosenberg, X. Song, J. Gao, S. Tiwary, R. Majumder, L. Deng, MS MARCO: A human generated machine reading comprehension dataset, in: Proceedings of the Workshop on Cognitive Computation: Integrating neural and symbolic approaches, volume 1773, 2016. URL: https://ceur-ws.org/Vol-1773/CoCoNIPS_2016_paper9.pdf.
[19] L. Delfino, D. Erriquez, S. Martinico, F. M. Nardini, C. Rulli, R. Venturini, kannolo: Sweet and smooth approximate k-nearest neighbors search, in: Proceedings of the 47th European Conference on Information Retrieval, 2025, pp. 400–406.
[20] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, M. Lomeli, L. Hosseini, H. Jégou, The FAISS library, arXiv preprint arXiv:2401.08281 (2024).
[21] Intel Corporation, Intel® oneAPI Math Kernel Library (oneMKL), 2024. URL: https://software.intel.com/en-us/mkl.
[22] B. Clavié, B. Warner, fastkmeans: Accelerated kmeans clustering in pytorch and triton, https://github.com/AnswerDotAI/fastkmeans/, 2025.
[23] T. Formal, C. Lassance, B. Piwowarski, S. Clinchant, From distillation to hard negative sampling: Making sparse neural ir models more effective, in: Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval, 2022, p. 2353–2359. doi:10.1145/3477495.3531857.